

# Teoría del solver dinámico FD3D (Madariaga)

Tal como está implementado en `legacy/Dynamic_inversion/` de KDEllipsPy

Documentación interna — KDEllipsPy (parte *Dynamic*)

6 de junio de 2026

## Resumen

Este documento describe, ecuación por ecuación y subrutina por subrutina, la física que resuelve el código dinámico legacy de KDEllipsPy: un solver de **ruptura espontánea 3D** por diferencias finitas en una malla escalonada velocidad–esfuerzo (Madariaga 1976; Virieux 1986), con fricción *slip-weakening* aplicada sobre la componente  $\sigma_{yz}$  de una falla plana, acoplado a las funciones de Green 1D de AXITRA (número de onda discreto) para sintetizar sismogramas. El objetivo es servir de base para reimplementar el *forward* en Python. Todas las afirmaciones están referidas a los fuentes en `Source/fd3d_subs/`.

## Índice

|                                                                      |          |
|----------------------------------------------------------------------|----------|
| <b>1. Panorama: la cadena del problema directo</b>                   | <b>1</b> |
| <b>2. Ecuaciones elastodinámicas: formulación velocidad–esfuerzo</b> | <b>1</b> |
| <b>3. Malla escalonada y operadores de diferencias finitas</b>       | <b>2</b> |
| 3.1. Malla escalonada (staggered grid)                               | 2        |
| 3.2. Operador espacial de 4. <sup>o</sup> orden                      | 2        |
| 3.3. Orden reducido en los bordes                                    | 2        |
| 3.4. Estabilidad (CFL)                                               | 3        |
| <b>4. Condiciones de borde del dominio</b>                           | <b>3</b> |
| 4.1. Borde absorbente (ABC)                                          | 3        |
| 4.2. Superficie libre                                                | 3        |
| <b>5. La falla: geometría, esfuerzos y fricción</b>                  | <b>3</b> |
| 5.1. Geometría de la falla en la malla                               | 3        |
| 5.2. Esfuerzo inicial: <code>mkstress</code>                         | 3        |
| 5.3. Esfuerzo de cedencia: <code>mkpeak</code>                       | 4        |
| 5.4. Ley de fricción slip-weakening y criterio de ruptura            | 4        |
| 5.5. Salidas de la dinámica                                          | 5        |
| <b>6. De slip-rate a sismogramas: seismograms + AXITRA</b>           | <b>5</b> |
| 6.1. Decimación de la fuente                                         | 5        |
| 6.2. Tensor de momento por subfalla: <code>cmoment</code>            | 5        |
| 6.3. Convolución en frecuencia y frecuencia compleja                 | 5        |
| 6.4. Filtrado de banda                                               | 5        |
| <b>7. Función de misfit: <code>calcmisfit</code></b>                 | <b>6</b> |
| 7.1. Rotación a R/T/Z                                                | 6        |
| 7.2. Ventanas por fase                                               | 6        |
| 7.3. Definición del misfit                                           | 6        |

|                                                     |   |
|-----------------------------------------------------|---|
| 8. Parámetros invertidos y algoritmo de inversión   | 6 |
| 9. Resumen de constantes y archivos clave           | 7 |
| 10. Implicancias para la reimplementación en Python | 7 |

## 1. Panorama: la cadena del problema directo

Una sola evaluación *forward* (subrutina `forward_modelling.f`) toma un vector de 10 parámetros del modelo `rmodel` y produce sismogramas sintéticos, en esta cadena:

1. `mkstress` — construye el campo de **esfuerzo inicial**  $\sigma^0(i, k)$  sobre la falla: una aspereza elíptica de alto esfuerzo más un círculo de nucleación.
2. `mkpeak` — construye el campo de **esfuerzo de cedencia** (*peak/yield stress*)  $\tau_p(i, k)$ .
3. `indyna3d (fd3d_theo.f)` — integra las ecuaciones elastodinámicas 3D por diferencias finitas con la condición de borde de fractura (fricción slip-weakening). Produce la historia de **slip-rate**  $\dot{s}(i, k, t)$  sobre la falla.
4. `seismograms (convn_dynsource.f)` — convoluciona el slip-rate con las funciones de Green de AXITRA y entrega los desplazamientos en las estaciones.
5. `calcmisfit` — compara con los datos observados (norma  $L_2$  rotada R/T/Z, ventanas P/S).

El bucle de inversión (Neighbourhood Algorithm, `NA_src/na.f`) llama repetidamente a esta cadena buscando los parámetros que minimizan el misfit.

## 2. Ecuaciones elastodinámicas: formulación velocidad–esfuerzo

El medio es elástico, lineal e isótropo. Las incógnitas son el campo de **velocidad de partícula**  $\mathbf{v} = (v_x, v_y, v_z)$  y el **tensor de esfuerzos**  $\boldsymbol{\sigma}$ . Las ecuaciones de gobierno son la conservación del momento (segunda ley de Newton para un continuo)

$$\rho \frac{\partial v_i}{\partial t} = \sum_j \frac{\partial \sigma_{ij}}{\partial x_j}, \quad (1)$$

y la ley constitutiva de Hooke en su forma de *tasa* (derivada temporal),

$$\frac{\partial \sigma_{ij}}{\partial t} = \lambda \delta_{ij} (\nabla \cdot \mathbf{v}) + \mu \left( \frac{\partial v_i}{\partial x_j} + \frac{\partial v_j}{\partial x_i} \right), \quad (2)$$

donde  $\rho$  es la densidad y  $\lambda, \mu$  las constantes de Lamé.

**Correspondencia con el código.** En el `common/disp/` de `parstat` los arreglos `u1, v1, w1` son las velocidades  $v_x, v_y, v_z$  (aunque los comentarios hablen de “displacement fields”): la actualización en `dvel` es exactamente (1). Los seis arreglos `xx, yy, zz, xy, yz, xz` son las componentes de  $\boldsymbol{\sigma}$ . Las propiedades del medio se guardan como

$$d1 = \rho, \quad mu1 = \mu = \rho v_s^2, \quad lam1 = \lambda = \rho (v_p^2 - 2v_s^2), \quad (3)$$

que es lo que arma `rdmd3d` al leer `model.dat` (un modelo 1D por capas,  $v_p, v_s, \rho$ , replicado en todo el plano horizontal).

### 3. Malla escalonada y operadores de diferencias finitas

#### 3.1. Malla escalonada (staggered grid)

El esquema es el clásico de Madariaga (1976), popularizado por Virieux (1986): las distintas componentes de  $\mathbf{v}$  y  $\boldsymbol{\sigma}$  viven en posiciones desplazadas medio paso  $\frac{1}{2}dh$  entre sí, y el tiempo se avanza en *leapfrog* (las velocidades en  $t + \frac{1}{2}dt$ , los esfuerzos en  $t + dt$ ). Esto da precisión de segundo orden en el tiempo con un solo nivel de almacenamiento. El paso espacial es uniforme,  $dh$ , e isótropo.

#### 3.2. Operador espacial de 4.º orden

Las derivadas espaciales usan el operador escalonado de cuarto orden, con coeficientes (`deriv.f`: `c1=9./8.`, `c2=-1./24.`)

$$\left. \frac{\partial f}{\partial x} \right|_i \approx \frac{1}{dh} \left[ c_1 (f_{i+\frac{1}{2}} - f_{i-\frac{1}{2}}) + c_2 (f_{i+\frac{3}{2}} - f_{i-\frac{3}{2}}) \right], \quad c_1 = \frac{9}{8}, \quad c_2 = -\frac{1}{24}. \quad (4)$$

En el código el medio-índice se materializa como diferencias entre nodos vecinos; por ejemplo, la actualización de  $v_x$  (`uxx1`) es

$$v_x^{t+\frac{1}{2}} = v_x^{t-\frac{1}{2}} + \frac{dt}{\rho dh} \left[ c_1 (\sigma_{xx}^i - \sigma_{xx}^{i-1}) + c_2 (\sigma_{xx}^{i+1} - \sigma_{xx}^{i-2}) + (\text{términos } xy, xz) \right], \quad (5)$$

réplica directa de (1) con el operador (4). Análogamente `vyy1`, `wzz1` avanzan  $v_y, v_z$ , y `dstres` avanza los esfuerzos según (2) con  $a = \lambda + 2\mu$ ,  $b = \lambda$  para las normales, y la media armónica/aritmética de  $\mu$  entre nodos vecinos (`xmu=xm1+xm2`) para las cizallas  $\sigma_{xy}, \sigma_{xz}, \sigma_{yz}$ .

#### 3.3. Orden reducido en los bordes

En la primera capa de nodos junto a cada cara se usa un operador de **segundo orden** (`uxx0/vyy0/wzz0` para velocidades vía `bnd2d`; `sxx, sxy0/syz0/sxz0` para esfuerzos vía `strbnd`), porque el operador de 4.º orden necesita dos nodos a cada lado.

#### 3.4. Estabilidad (CFL)

La condición de estabilidad del esquema explícito es de tipo Courant. El código la reporta como (`wrpm3d`, formato 70):

$$\text{CFL} = \frac{v_p^{\text{máx}} dt}{dh} < 0,5. \quad (6)$$

$v_p^{\text{máx}}$  es la mayor velocidad  $P$  del modelo. Hay que elegir  $dt$  acorde a  $dh$  y al modelo para no violar (6).

### 4. Condiciones de borde del dominio

#### 4.1. Borde absorbente (ABC)

Para que la malla finita no refleje energía hacia adentro, `abc` aplica una condición **paraxial de primer orden** (Clayton–Engquist) en las seis caras. Para una onda que sale normal a una cara, la actualización tiene la forma

$$f^{\text{borde}} += \frac{c dt}{dh} (f^{\text{interior}} - f^{\text{borde}}), \quad c = \sqrt{\frac{\lambda+2\mu}{\rho}} = v_p \text{ (normal)}, \quad c = \sqrt{\frac{\mu}{\rho}} = v_s \text{ (tangencial)}, \quad (7)$$

donde se usa  $v_p$  para la componente normal a la cara y  $v_s$  para las tangenciales. Es una ABC sencilla (la actualización `FD3D_TSN` la reemplaza por PML, mucho más eficaz).

## 4.2. Superficie libre

Existen rutinas de superficie libre (**fres** para esfuerzos:  $\sigma_{zz}, \sigma_{xz}, \sigma_{yz} = 0$  en  $z = n_z$  vía imágenes antisimétricas; **fuvw** para velocidades). En la configuración que nos interesa (fuente profunda,  $\sim 123$  km) el bucle principal de **indyna3d** usa ABC en todas las caras; la superficie libre del semiespacio real la aporta AXITRA al sintetizar.

## 5. La falla: geometría, esfuerzos y fricción

### 5.1. Geometría de la falla en la malla

La falla es un **plano** ubicado en  $j = n_{y_{sc}}$  (índice fijo en  $y$ ); su **normal** es  $\hat{y}$ . El deslizamiento (slip) es en dirección  $z$ , es decir **dip-slip** (comentario en **fd3d\_theo.f**: “modified september 2011 for dip slip”). La tracción de cizalla relevante sobre el plano es  $\sigma_{yz}$ , y la fricción se impone sobre *esa* componente.

**Modelo de media falla (antisimetría).** Sólo se simula la mitad del medio: la velocidad de deslizamiento se toma como  $\dot{s} = 2v_z(i, n_{y_{sc}}+1, k)$  y el slip acumulado y el momento llevan el factor 2 (líneas 2.\*w1, 2.\*slip en **indyna3d** y **wrss3d**). Esto explota la antisimetría del campo a través del plano de falla.

### 5.2. Esfuerzo inicial: mkstress

**mkstress** llena el campo de tracción inicial de cizalla  $\sigma^0(i, j) \equiv \mathbf{strinix}$  sobre el plano. Para cada punto  $(i, j)$  se evalúa su posición en el marco propio de la elipse (centro  $(x_o, y_o)$ , rotada un ángulo  $\phi$ ):

$$x' = (i - x_o) \cos \phi + (j - y_o) \sin \phi, \quad (8)$$

$$y' = -(i - x_o) \sin \phi + (j - y_o) \cos \phi, \quad r_e = \sqrt{\left(\frac{x'}{a}\right)^2 + \left(\frac{y'}{b}\right)^2}. \quad (9)$$

El punto está *dentro de la aspereza* si  $r_e < 1$ . Entonces

$$\sigma^0(i, j) = \begin{cases} \sigma_{\text{in}} (= \mathbf{strbin} = T_e \cdot 10^6 \text{ Pa}), & r_e < 1 \quad (\text{aspereza}), \\ \sigma_{\text{out}} (= -10^8 \text{ Pa}), & r_e \geq 1 \quad (\text{fuera}). \end{cases} \quad (10)$$

El valor exterior fuertemente negativo confina la ruptura a la aspereza (fuera no puede romper). Además, dentro de un **círculo de nucleación** de radio  $r$  centrado en el hipocentro  $(x_a, y_a) = (n_x/2, n_y/2)$  se *suma* un sobre-esfuerzo  $\sigma_{\text{nuc}}$  (**strain**) para forzar el inicio espontáneo:

$$\text{si } (i - x_a)^2 + (j - y_a)^2 \leq r^2 : \quad \sigma^0(i, j) += \sigma_{\text{nuc}}. \quad (11)$$

### 5.3. Esfuerzo de cedencia: mkpeak

**mkpeak** arma el campo de **esfuerzo pico** (yield)  $\tau_p(i, j) \equiv \mathbf{peak\_xz}$ . En el código ambas ramas (dentro/fuera de la elipse) asignan el mismo valor **strin**, de modo que el esfuerzo pico resulta **uniforme** sobre la falla:

$$\tau_p(i, j) = \tau_p \equiv \mathbf{strin} = c_1 \cdot \sigma_{\text{in}} = c_1 T_e \cdot 10^6 \text{ Pa}, \quad (12)$$

donde  $c_1 = \mathbf{rmodel}(7)$  (parámetro **cte1**). Es decir,  $\tau_p$  es un múltiplo del esfuerzo inicial de la aspereza.

#### 5.4. Ley de fricción slip-weakening y criterio de ruptura

En cada paso de tiempo, tras actualizar los esfuerzos, se aplica la **condición de borde de fractura** en el plano  $j = n_{ysc}$  (`indyna3d`). Se trabaja con la tracción *absoluta*

$$\tau_{\text{abs}}(i, k) = \underbrace{\sigma_{yz}(i, n_{ysc}, k)}_{\text{relativa, FD}} + \underbrace{\sigma^0(i, k)}_{\text{inicial}}. \quad (13)$$

**(a) Criterio de iniciación.** Un punto aún intacto rompe cuando su tracción absoluta supera el esfuerzo pico:

$$\tau_{\text{abs}}(i, k) > \tau_p(i, k) \implies \text{punto roto}, \quad t_{\text{rup}}(i, k) = t. \quad (14)$$

**(b) Ley slip-weakening.** Una vez roto y deslizando, la tracción cae linealmente con el slip acumulado  $s$  hasta una distancia crítica  $D_c$  (*slip-weakening* clásico de Ida/Andrews; aquí en su versión Madariaga):

$$\tau(i, k) = \tau_p(i, k) \cdot \begin{cases} 1 - \frac{s}{D_{\text{máx}}}, & s \leq D_{\text{máx}}, \\ 0, & s > D_{\text{máx}}, \end{cases} \quad (15)$$

y se impone como condición de borde escribiendo en el campo relativo  $\sigma_{yz}(i, n_{ysc}, k) = \tau(i, k) - \sigma^0(i, k)$ . En el código  $D_{\text{máx}} = \text{rmodel}(10)$  es  $D_c/2$  (consistente con el factor 2 de la media falla). El slip se integra explícitamente:

$$s(i, k) += v_z(i, n_{ysc}+1, k) dt. \quad (16)$$

**(c) Cierre/no-penetración.** Si la apertura normal se vuelve negativa ( $v_z(i, n_{ysc}+1, k) < 0$ , la falla tiende a cerrarse), el punto se “des-rompe” y se anulan las velocidades  $v_x, v_z$  en el plano, evitando interpenetración.

#### 5.5. Salidas de la dinámica

Al terminar el bucle temporal, `indyna3d` escribe (si `ioutput=1`) los mapas finales sobre la falla: tiempo de ruptura  $t_{\text{rup}}$  (`ruptime.res`), slip final  $2s$  (`slip.res`) y caída de esfuerzo  $\sigma_{yz}$  (`stressdrop.res`). El momento sísmico (sin escalar por  $\mu$  y área aún) se acumula como

$$\text{amom} = \sum_{i,k} 2s(i, k). \quad (17)$$

### 6. De slip-rate a sismogramas: seismograms + AXITRA

La radiación se calcula con el **teorema de representación**: el campo en una estación es la convolución del slip-rate de cada subfalla con la función de Green del medio 1D. Las Green las provee AXITRA (número de onda discreto, Bouchon 1981) y vienen pre-calculadas en `axi.res` (léidas por `readaxires` en los arreglos `uuxf, uuyf, uuzf` indexados por frecuencia, fuente, receptor y las 6 dislocaciones elementales).

#### 6.1. Decimación de la fuente

La malla dinámica (fina,  $n_x \times n_y$ ) se **decima** por un factor `idec=4` promediando bloques  $4 \times 4$ , de modo que la malla de fuentes para la radiación es  $4 \times$  más gruesa que la del solver FD. Esto reduce el número de fuentes puntuales en la convolución sin perder la cinemática gruesa.

## 6.2. Tensor de momento por subfalla: `cmoment`

Cada subfalla es un **doble par** (dislocación de cizalla) con orientación (strike, dip, rake). `cmoment` convierte el mecanismo en los seis coeficientes  $a_{1..6}$  de las dislocaciones elementales que AXITRA sabe propagar. El momento escalar por subfalla es  $M_0 = \mu \cdot \text{área}$  (el slip ya entra después vía el espectro de slip-rate), con  $\mu = \rho v_s^2$  evaluado en la capa donde cae la fuente. Los coeficientes son las combinaciones estándar de sin/cos de strike, dip y rake (ecuaciones `x1..x6`  $\rightarrow$  `a(1..6)`).

## 6.3. Convolución en frecuencia y frecuencia compleja

El slip-rate de cada subfalla  $\dot{s}(i, j, t)$  se transforma a frecuencia con una FFT. Para suprimir el *wraparound* temporal (artefacto de la transformada discreta) se usa una **frecuencia compleja** con parte imaginaria

$$\omega = 2\pi f - i a_w / (\text{escala}), \quad a_w = -\pi \text{aw} / t_l, \quad (18)$$

lo que equivale a multiplicar la señal por  $e^{-a_w t}$  antes de transformar y deshacerlo al final (técnica de Bouchon/Kennett). El desplazamiento espectral en el receptor  $r$  es la suma sobre subfuentes  $s$  y sobre las 6 dislocaciones:

$$u_c(\omega, r) = \sum_s \left[ \sum_{m=1}^6 G_m^c(\omega, s, r) a_m(s) \right] \dot{S}(\omega, s), \quad c \in \{x, y, z\}, \quad (19)$$

donde  $\dot{S}(\omega, s)$  es el espectro del slip-rate de la subfalla  $s$  y  $G_m^c$  las Green de AXITRA. Una FFT inversa devuelve las trazas en tiempo y se deshace el peso exponencial de la frecuencia compleja.

## 6.4. Filtrado de banda

Cada traza sintética se filtra con un pasabanda Butterworth (`XAPIIR`, tipo BU, BP) entre `fr1` y `fr2` (típicamente 0,02–0,1 Hz). El resultado se vuelca a `predicted_data` en el orden  $(x, y, z)$  por estación, que es el que espera el misfit. Si `ioutput=1` también se escriben `dyn_disp_x/y/z`.

## 7. Función de misfit: `calcmisfit`

El misfit es una **norma  $L_2$  normalizada**, calculada sobre componentes **rotadas** radial/transversal/vertical (R/T/Z) y **ventaneada** por fases. Es idéntico en filosofía al misfit cinemático de KDEllipsPy.

### 7.1. Rotación a R/T/Z

Con el acimut  $\phi$  de cada estación (de `azi_times.txt`), las componentes N ( $x$ ) y E ( $y$ ) se rotan:

$$R = N \cos \phi + E \sin \phi, \quad T = E \cos \phi - N \sin \phi, \quad Z = Z. \quad (20)$$

### 7.2. Ventanas por fase

- Onda **P**: se evalúan las componentes **radial** y **vertical** en una ventana que arranca en el tiempo de arribo  $t_P$  y dura `time_window` = 20 s.
- Onda **S(H)**: se evalúa la componente **transversal** en una ventana que arranca en  $t_S$  y dura 20 s.

El muestreo de trabajo del misfit es 4 Hz, así que el índice de inicio es `nint`( $t_{P/S}$ )  $\times$  4.

### 7.3. Definición del misfit

Acumulando sobre componentes, estaciones y muestras dentro de las ventanas:

$$\text{misfit} = \frac{\sum (d^{\text{obs}} - d^{\text{pred}})^2}{\sum (d^{\text{obs}})^2} = \frac{\text{num}}{\text{den}}. \quad (21)$$

Si la ruptura **no nuclea**, las sintéticas son nulas,  $\text{num} = \text{den}$  y el misfit da exactamente 1,0; el código lo detecta y lo manda a  $\infty$  ( $\text{misfitval}/0$ ) para que el NA descarte ese modelo.

## 8. Parámetros invertidos y algoritmo de inversión

El forward depende de **10 parámetros dinámicos** (`rmodel(1..10)` en `forward_modelling.f`; rangos en `input_dynamic_ellipse.txt`):

| #                       | Símbolo / significado                                                       | Unidad          |
|-------------------------|-----------------------------------------------------------------------------|-----------------|
| <code>rmodel(1)</code>  | $a$ — semieje 1 de la aspereza elíptica                                     | puntos de malla |
| <code>rmodel(2)</code>  | $b$ — semieje 2 de la aspereza                                              | puntos de malla |
| <code>rmodel(3)</code>  | $x_o$ — centro de la elipse (strike)                                        | puntos de malla |
| <code>rmodel(4)</code>  | $y_o$ — centro de la elipse (dip)                                           | puntos de malla |
| <code>rmodel(5)</code>  | $\phi$ — ángulo de rotación de la elipse                                    | radianes        |
| <code>rmodel(6)</code>  | $T_e$ — esfuerzo inicial de la aspereza ( $\sigma_{\text{in}} = T_e 10^6$ ) | MPa             |
| <code>rmodel(7)</code>  | $c_1$ — $\tau_p = c_1 \sigma_{\text{in}}$ (esfuerzo pico)                   | adim.           |
| <code>rmodel(8)</code>  | $c_2$ — $\sigma_{\text{nuc}} = c_2 \tau_p$ (sobre-esfuerzo de nucleación)   | adim.           |
| <code>rmodel(9)</code>  | $r$ — radio del círculo de nucleación                                       | puntos de malla |
| <code>rmodel(10)</code> | $D_{\text{máx}} = D_c/2$ — distancia crítica slip-weakening                 | m               |

Cuadro 1: Los 10 parámetros del modelo dinámico. El hipocentro está fijo en el centro de la malla  $(x_a, y_a) = (n_x/2, n_y/2)$ ; `strbout` =  $-10^8$  Pa y `straout` = 0 están *hardcodeados*.

**Neighbourhood Algorithm (NA).** El bucle externo (`Source/NA_src/na.f`; Sambridge 1999) muestrea el espacio de 10 parámetros, evalúa el misfit (21) de cada modelo vía la cadena forward, y re-muestrea las celdas de Voronoi de menor misfit. Los controles (número de iteraciones, tamaño de muestra inicial y por iteración, celdas a re-muestrear, número de núcleos) salen de `input_dynamic_ellipse.txt`. `fd3d_direct` corre un único forward del mejor modelo con `ioutput=1` para volcar todas las salidas.

## 9. Resumen de constantes y archivos clave

- **Malla FD** (`parstat`):  $n_x=n_y=n_z=160$  (en compilación). La malla de fuentes radiantes es  $n_x/4 \times n_y/4$ .
- **Operador 4.<sup>o</sup> orden**:  $c_1 = 9/8$ ,  $c_2 = -1/24$ .
- **Estabilidad**:  $v_p^{\text{máx}} dt/dh < 0,5$ .
- **Fricción**: slip-weakening sobre  $\sigma_{yz}$ ,  $\tau_p$  uniforme,  $D_{\text{máx}} = D_c/2$ .
- **Green**: AXITRA 1D (número de onda discreto), 6 dislocaciones elementales, frecuencia compleja  $a_w$  para anti-wraparound.
- **Banda**: pasabanda Butterworth XAPIIR  $[fr_1, fr_2]$ .

- **Misfit:**  $L_2$  normalizado, R/T/Z, ventanas P (R,Z) y S (T) de 20 s desde  $t_P/t_S$ , muestreo 4 Hz.
- **I/O de la falla:** `stressin.dat`, `peakin.dat`, `ruptime.res`, `slip.res`, `stressdrop.res`, `sliprate.res`, `synth.res`, `dyn_disp_{x,y,z}`.

## 10. Implicancias para la reimplementación en Python

Para portar el *forward* a Python conviene separar tres bloques:

1. **Generador ellipse**  $\rightarrow$  **campos** ( $\sigma^0, \tau_p$ ): equivale a `mkstress`+`mkpeak`; es álgebra vectorizada trivial en NumPy y reutiliza la misma parametrización elíptica del lado cinemático.
2. **Solver FD3D** (`indyna3d`): es el núcleo costoso. Opciones: (a) envolver el Fortran existente con `f2py` (igual que ya se hizo con `AXITRA`), o (b) adoptar **FD3D\_TSN** (Premus–Gallovič), que es el descendiente directo del mismo solver de Madariaga con *traction-at-split-node*, PML y aceleración GPU. La salida que se necesita es la historia de slip-rate  $\dot{s}(i, k, t)$ .
3. **Radiación + misfit**: la convolución con `AXITRA` ya está envuelta en `kdellipsy/axitra`, y el misfit (21) coincide con el cinemático ya implementado. El NA en Python (`NAInversionModel`) puede orquestar todo, sustituyendo el frágil `bash+sed` del legacy.